



Enseignante : Pr. Marie NDIAYE DIOP

Généralités

DataFrame

Un DataFrame est un ensemble de données organisé en colonnes nommées. Il est conceptuellement équivalent à une table dans une base de données relationnelle ou à un data frame dans R/Python, mais avec des optimisations plus riches. Les DataFrames peuvent être construits à partir d'un large éventail de sources telles que : des fichiers de données structurés, des tables dans Hive, des bases de données externes ou des RDDs existants.

Dataset

Un Dataset est une collection distribuée de données. **Dataset** est une nouvelle interface ajoutée dans Spark 1.6 qui offre les avantages des RDD (typage fort, possibilité d'utiliser de puissantes fonctions lambda) avec les avantages du moteur d'exécution optimisé de Spark SQL. Un Dataset peut être construit à partir d'objets JVM, puis manipulé à l'aide de transformations fonctionnelles (**map**, **flatMap**, **filter**, etc.).

Préambule

Télécharger le fichier **people.json** à partir de l'espace du TP dans Google Classroom, ce fichier contient des informations sur des personnes au format JSON.

JSON est l'abréviation de JavaScript Object Notation. Il s'agit d'un format léger pour le stockage et le transport de données. JSON est souvent utilisé lorsque des données sont envoyées d'un serveur à une page Web. Il est "autodescriptif" et facile à comprendre.

L'exemple ci-dessous définit un objet "employees" : un tableau de 3 enregistrements d'employés (objets) :

```
{
  "employees": [
    {"firstName": "John", "lastName": "Doe"},
    {"firstName": "Anna", "lastName": "Smith"},
    {"firstName": "Peter", "lastName": "Jones"}
  ]
}
```

Exercice 1 : Création d'un projet Java Spark SQL

1. A partir de Eclipse, créer un projet Maven comme cela a été fait lors du TP précédent.
2. Dans le fichier pom.xml, ajouter le contenu ci-dessous.

Propriétés pour le compilateur

```
<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
```



Enseignante : Pr. Marie NDIAYE DIOP

```
<maven.compiler.target>1.8</maven.compiler.target>
</properties>
```

Dépendances pour les librairies de Spark requis dans le projet

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.1.3</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.apache.spark/spark-sql -->
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.1.3</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Spécification du chemin de la classe contenant la méthode main

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>sparksql.Main</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

3. Créer une classe Main contenant la méthode main.

Exercice 2 : Création d'un Dataframe

<https://spark.apache.org/docs/latest/sql-programming-guide.html>

1. Créer une nouvelle session
 - a. Créer un nouvel objet de type **SparkSession.Builder** (classe **SparkSession**, méthode **builder()**).



Enseignante : Pr. Marie NDIAYE DIOP

- b. A l'aide de cet objet, initialiser le nom de l'application (class **SparkSession.Builder**, méthode **appName(String name)**).
 - c. Créer la session – objet de type **SparkSession** (class **SparkSession.Builder**, méthode **getOrCreate()**)
2. Charger les données dans un dataframe à partir du fichier **people.json**
 - a. Charger les données JSON en appelant les méthodes **read().json(String path)** sur l'objet de type **SparkSession** créé précédemment. Ces méthodes permettent de charger des données à partir de sources externes (**read()** de type JSON (**json(String path)**)). L'instruction retourne un dataframe, c'est-à-dire un objet de type **Dataset<Row>**.

Exercice 3 : Sélection de données à partir d'un Dataframe

Les opérations que vous effectuerez par la suite se feront sur le dataframe créé dans l'exercice 2.

1. Afficher le schéma des données (classe **Dataset<Row>**, méthode **printSchema()**).
2. Afficher les enregistrements du dataset (classe **Dataset<Row>**, méthode **show()**). Si les enregistrements sont nombreux, seuls les 20 premiers sont affichés.
3. Sélectionner et afficher les noms/*name* (classe **Dataset<Row>**, méthodes **select(String col, String... cols)** et **select(Column... cols)**).
4. Sélectionner et afficher les noms/*name* et les villes/*city* (classe **Dataset<Row>**, méthode **select(String col, String... cols)** ou **select(Column... cols)**).
5. Sélectionner et afficher les personnes habitant à Marvinville (classe **Dataset<Row>**, méthodes **filter(String conditionExpr)** puis **select(String col, String... cols)** ou **select(Column... cols)**).
6. Sélectionner et afficher le nombre de personnes par ville/*city* (classe **Dataset<Row>**, méthodes **groupBy(Column... cols)** ou **groupBy(String col1, String... cols)** puis **count()**).

Exercice 4 : Insertion de données dans un Dataframe

En vous aidant de la documentation Java de Spark, insérer de nouvelles données dans le dataframe.

Indication : voir la méthode **sql(String sqlText)** de la classe **SparkSession**.

Exercice 5 : Dataframe et RDD

Charger dans un JavaRDD le dataframe créé dans l'exercice 2.

Références

- <https://gist.github.com/raine/da15845f332a2fb8937b344504abfbc0>
- <https://github.com/apache/spark/blob/master/examples/src/main/resources/people.json>
- <https://mvnrepository.com/artifact/org.apache.spark/spark-sql>
- <https://spark.apache.org/docs/2.1.0/sql-programming-guide.html>
- <https://sparkbyexamples.com/>